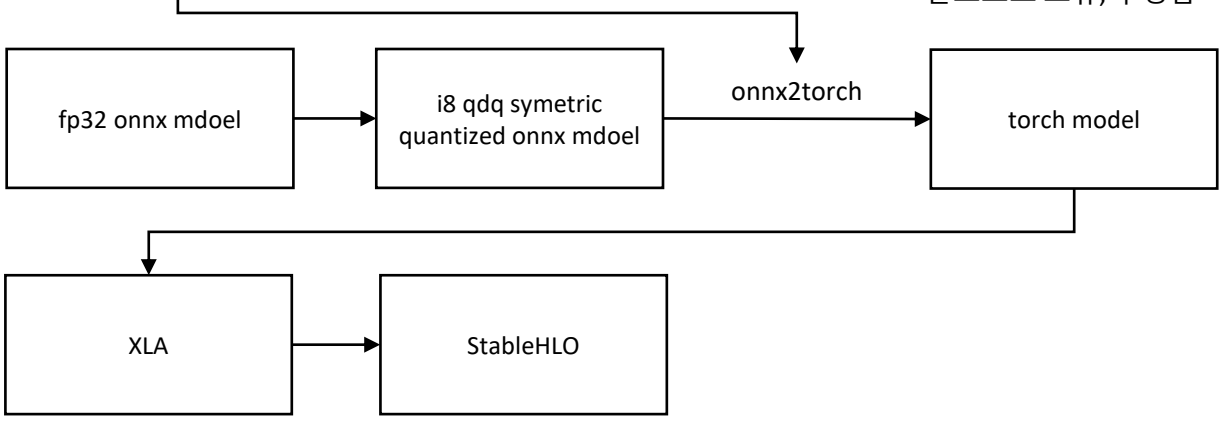
New Pipeline (260331 new)

onnx2torch custom definition

- OnnxDequantizeLinear, OnnxQuantizeLinear → onnx2torch에 Conversion이 구현되어 있지 않아, 직접 구현하여 등록
- Split, Reshape, Resize, Slice, TopK, Tile, Unsqueeze → Dynamic Shape로 Conversion 되도록 구현되어 있어서 Static으로 Override 시킴
- conv2d → weight bias등을 반드시 initialize에서 받게 되어 있는데, qdq에서는 quantizer의 result를 받으므로 오류, 수정함



StableHLO Layer

InlineConstants Pass	arguments로 있는 meta data를 inline화
Canonicalizer Pass	code 정리
CSE Pass	
IdentityElim Pass	sub(x,0), add(x,0)등 의미없는 연산 제거
Canonicalizer Pass	code 정리
CSE Pass	
MathToQDQ Pass	산술 연산으로부터 Quantize, Dequantize 추출 (fuse)
StablehloLegalizeQDQToQuantized Op Pass	q - fp32 op - dq를 i8 qop로 변환
QuantizeRegion Pass	i8/fp32 연산영역 구별, quant type으로 rewire
QuantCleanup Pass	noop quantize, dequantize제거 i8 concat fusion

Custom IR Layer

InlineConstants Pass	

SCF / Linalg / Affine IR Layer

HW IR Layer

Serialize Layer

Workload (260309 new)

custom flow dispatch region	260205

1순위 구현	완성
2순위 구현	미숙
3순위 구현	TODO

INF_CAP Layer (infetron-capability layer)

IREE to CAP PP

- QDQFusion Pass
- OpFusion Pass
- PostOpFusion Pass
- MonoOpFusion Pass
- VailPad Pass
- QOpFusion Pass
- GenericOpen Pass
- ExitIREE Pass

INF_ISA Layer (infetron-ISA layer)

ISA의 opcode를 중심으로 제작

CAP to ISA PP

- CoreDistribution Pass — A 코어용 연산과 P코어용 연산을 나눔 p코어로만 둘 수도 있음.
- FitToCore Pass — 데이터 포맷에 맞게 연산을 조정 (대표적으로 tiling)

Structural Scheduling PP

- RecordLatency Pass — walk 사용) op별 latency 기록
- SetOpPriority Pass — walk, c++로 알고리즘) concat을 기준으로 region을 나눔 각 region에서 critical path를 찾아, 우선순위 1순위 부여
- CoreArrange Pass — core에 연산을 할당. 아래의 우선순위로 연산 할당
  1. critical path에 따른 우선순위에 따라 먼저 배치
  2. 연산이 긴 것을 노는 자리에 배치
  3. data locality를 보고 연산을 배치
- CoreRearrange Pass — region과 region 사이 연결지점에서 data locality를 보고 core에 할당한 연산 묶음을 재배치
- CreateStall Pass — virtual stall 삽입. 계산에만 사용하고 최종 바이너리에서 stall은 삭제. stall을 확인하고 최적화를 하기 위해 추가함

DataInfo PP

- CreateDataMoveInfo Pass — 데이터 이동정보 (move : UB간 움직임, load/store: SM <-> UB 간 데이터 이동)를 나타내는 ops 추가 데이터의 위치를 타이트하게 관리

Data-Flow Aligner PP

- CopyOp Pass

INF_EXP Layer (infetron-Export layer)

Core Bin Export PP

- DestroyStall Pass — stall은 최적화에 이미 모두 활용되었으므로 삭제
- CreateFinal ISA Pass — 주변 ops들을 결합하며 문서에서 정의하는 ISA와 구조를 완벽히 일치, 부족한 정보가 있을 경우 앞쪽에 필요한 Pass를 추가로 정의
- ISA BinGenerate Pass — ISA를 core별로 .cbin으로 저장

Weight Bin Export PP

- WeightBufferization Pass — arith.constant를 보고 SM에 저장할 weights를 정리, .wbin으로 저장

Plan Export PP

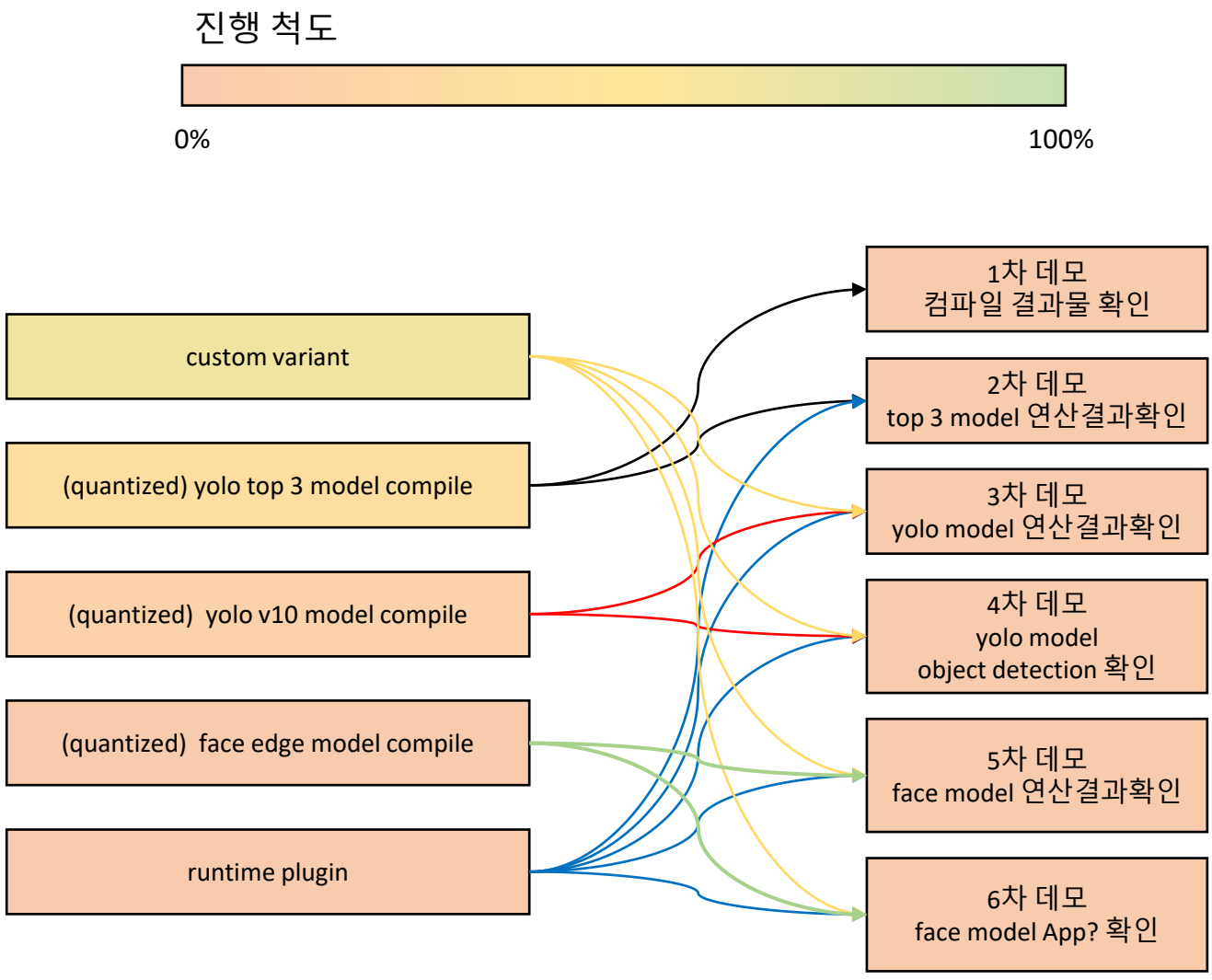
- CreatePlan Pass — Plan.json 생성

결과물

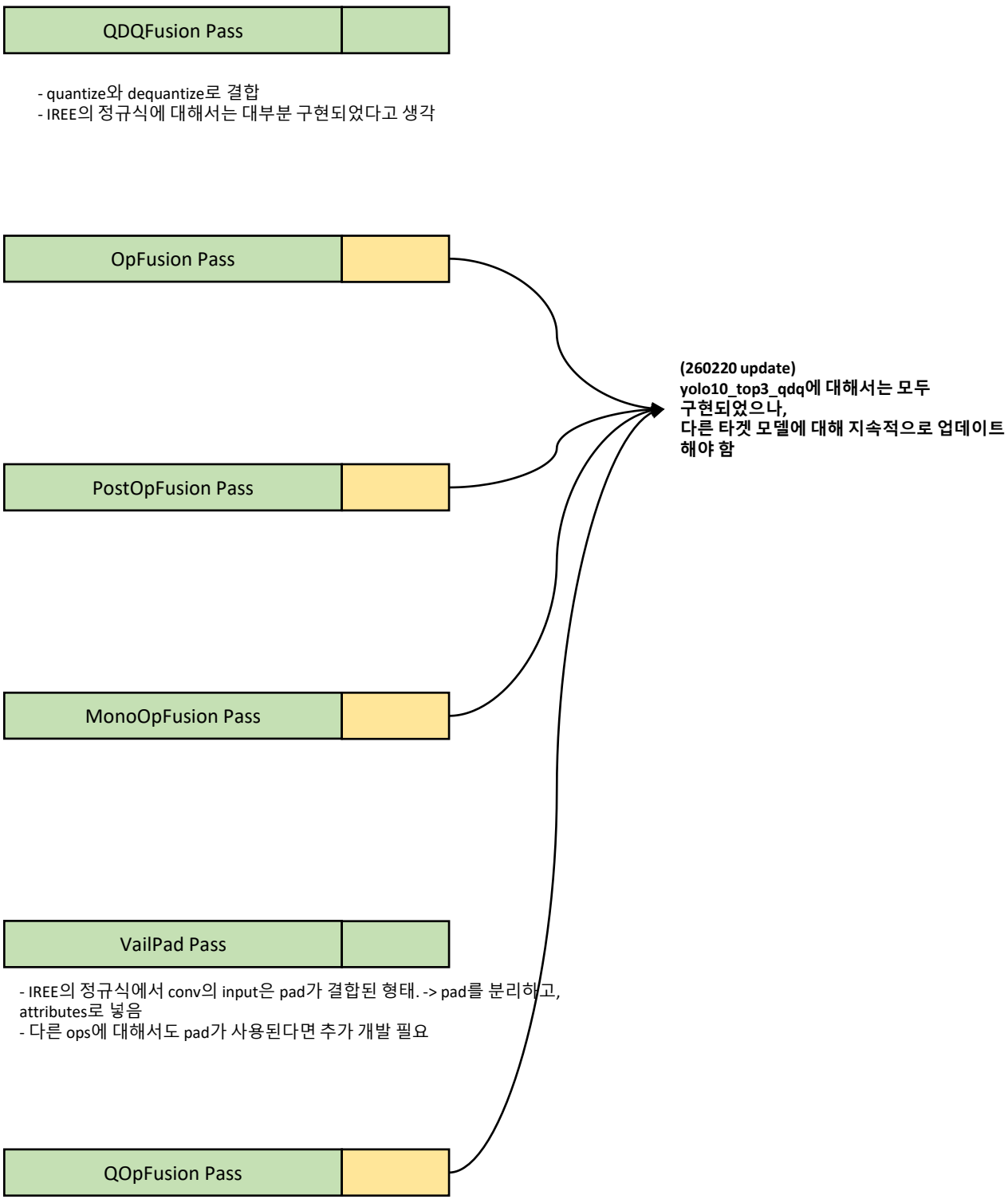
- binary : out_bin.ihnn
- plan : plan.json
- out.mlir => sub tool을 결합하여 profile + Debug 파일 생성

Runtime 제작

완성도


진행 척도
0%
100%
custom variant
(quantized) yolo top 3 model compile
(quantized) yolo v10 model compile
(quantized) face edge model compile
runtime plugin
1차 데모
컴파일 결과물 확인
2차 데모
top 3 model 연산결과확인
3차 데모
yolo model 연산결과확인
4차 데모
yolo model
object detection 확인
5차 데모
face model 연산결과확인
6차 데모
face model App? 확인

IREE to CAP PassPipeline



GenericOpen Pass

- Generic을 제거하며 내부 연산식을외부로 끌어냄.
- 조건 : generic 내부의 Ops가 input output에 대해 scalar와 tensor 모두 포함할 수 있어야 함.
- 이 패스는 더 수정할 필요가 없지만, generic을 사용할 수 있도록 앞선 패스들을 조정해야 할 필요는 있음

ExitIREE Pass

- 일부만 구현 됨.

구현 목록	개발척도
1순위 구현	완성
2순위 구현	최소구현
3순위 구현	TODO

Workload (260227 old)

custom flow dispatch region	260205

1순위 구현	완성
2순위 구현	미숙
3순위 구현	TODO

INF_CAP Layer (infetron-capability layer)

IREE to CAP PP

- QDQFusion Pass
- OpFusion Pass
- PostOpFusion Pass
- MonoOpFusion Pass
- VailPad Pass
- QOpFusion Pass
- GenericOpen Pass

pcore랑 acore는 pair라고 생각하고 진행
-> pcore와 acore가 병렬로 컴퓨팅을 할 것
같지는 않음. 병렬로 하더라도 규칙이 명확함

- To ACore Pass
- FitToCore Pass

데이터 포맷에 맞게 연산을 조정 (대표적으로 tiling)

INF_ISA Layer (infetron-ISA layer)

ISA의 opcode를 중심으로 제작

CAP to ISA PP

- To PCore Pass

Structural Scheduling PP

- RecordLatency Pass — walk 사용) op별 latency 기록
- SetOpPriority Pass — walk, c++로 알고리즘) concat을 기준으로 region을 나눔 각 region에서 critical path를 찾아, 우선순위 1순위 부여
- CoreArrange Pass — core에 연산을 할당. 아래의 우선순위로 연산 할당
  1. critical path에 따른 우선순위에 따라 먼저 배치
  2. 연산이 긴 것을 노는 자리에 배치
  3. data locality를 보고 연산을 배치
- CoreRearrange Pass — region과 region 사이 연결지점에서 data locality를 보고 core에 할당한 연산 묶음을 재배치
- CreateStall Pass — virtual stall 삽입. 계산에만 사용하고 최종 바이너리에서 stall은 삭제. stall을 확인하고 최적화를 하기 위해 추가함

DataInfo PP

- CreateDataMoveInfo Pass — 데이터 이동정보 (move : UB간 움직임, load/store: SM <-> UB 간 데이터 이동)를 나타내는 ops 추가 데이터의 위치를 타이트하게 관리

Data-Flow Aligner PP

- CopyOp Pass

INF_EXP Layer (infetron-Export layer)

Core Bin Export PP

- DestroyStall Pass — stall은 최적화에 이미 모두 활용되었으므로 삭제
- CreateFinal ISA Pass — 주변 ops들을 결합하며 문서에서 정의하는 ISA와 구조를 완벽히 일치, 부족한 정보가 있을 경우 앞쪽에 필요한 Pass를 추가로 정의
- ISA BinGenerate Pass — ISA를 core별로 .cbin으로 저장

Weight Bin Export PP

- WeightBufferization Pass — arith.constant를 보고 SM에 저장할 weights를 정리, .wbin으로 저장

Plan Export PP

- CreatePlan Pass — Plan.json 생성

결과물
- binary : out_bin.ihnn
- plan : plan.json
- out.mlir => sub tool을 결합하여 profile + Debug 파일 생성

Runtime 제작

Workload (260225 old)

custom flow dispatch region	260205

1순위 구현	완성
2순위 구현	미숙
3순위 구현	TODO

INF_CAP Layer (infetron-capability layer)

IREE to CAP PP

- QDQFusion Pass
- OpFusion Pass
- PostOpFusion Pass
- MonoOpFusion Pass
- VailPad Pass
- QOpFusion Pass
- GenericOpen Pass

pcore랑 acore는 pair라고
-> pcore와 acore가 병렬로
같지는 않음. 병렬로 하다

- To ACore Pass
- FitToCore Pass

데이터 포맷에 맞게 연산을 조

??

INF_ISA Layer (infetron-ISA layer)

ISA의 opcode를 중심으로 제작

CAP to ISA PP

- To PCore Pass

Optimization PP

- RecordLatency Pass
- SetOpPriority Pass
- CoreArrange Pass
- CoreRearrange Pass

walk 사용) op별 latency 기록

walk, c++로 알고리즘) concat을 기준으로 region을 나눔
각 region에서 critical path를 찾아, 우선순위 1순위 부여

core에 연산을 할당. 아래의 우선순위로 연산 할당
1. critical path에 따른 우선순위에 따라 먼저 배치
2. 연산이 긴 것을 노는 자리에 배치
3. data locality를 보고 연산을 배치

??

region과 region 사이 연결지점에서 data locality를 보고
core에 할당한 연산 묶음을 재배치

DataInfo PP

- CreateDataMoveInfo Pass

데이터 이동정보 (move : UB간 움직임, load/store: SM <-> UB 간 데이터 이동)를 나타내는 ops 추가
데이터의 위치를 타이트하게 관리

INF_EXP Layer (infetron-Export layer)

Core Bin Export PP

- CreateFinal ISA Pass
- ISA BinGenerate Pass

주변 ops들을 결합하며 문서에서 정의하는 ISA와 구조를 완벽히 일치, 부족한 정보가 있을 경우 앞쪽에 필요한 Pass를 추가로 정의

ISA를 core별로 .cbin으로 저장

Weight Bin Export PP

- WeightBufferization Pass

arith.constant를 보고 SM에 저장할 weights를 정리, .wbin으로 저장

Plan Export PP

- CreateHAL Pass
- CreatePlan Pass

??

Custom HAL 생성, -> 필요한가? Plan과 Runtime이 어떻게 되는지에 따라 필요없을 수도

Plan.json 생성

결과물
- core별 binary : .cbin
- meta data binary : .wbin
- plan : plan.json(?)

Runtime 제작

Workload (old)

custom flow dispatch region	260205

1순위 구현	완성
2순위 구현	미숙
3순위 구현	TODO

INF_CAP Layer (infetron-capability layer)

IREE to CAP PP

- QDQFusion Pass
- OpFusion Pass
- PostOpFusion Pass
- MonoOpFusion Pass
- VailPad Pass
- QOpFusion Pass
- GenericOpen Pass

INF_ISA Layer (infetron-ISA layer)

CAP to ISA PP

- To NCore Pass
- To APCore Pass

PCore Packing PP

- PCore VLIW Packing Pass
- PCore Order Free Packing Pass

A core BlackBox

PCore Packing PP

- PCore OrderingAndMarking Pass
- PCore Spreading Pass
- PCore Folding Pass
- PCore ISA Canonicalization Pass

INF_Q Layer (infetron-Queue layer)

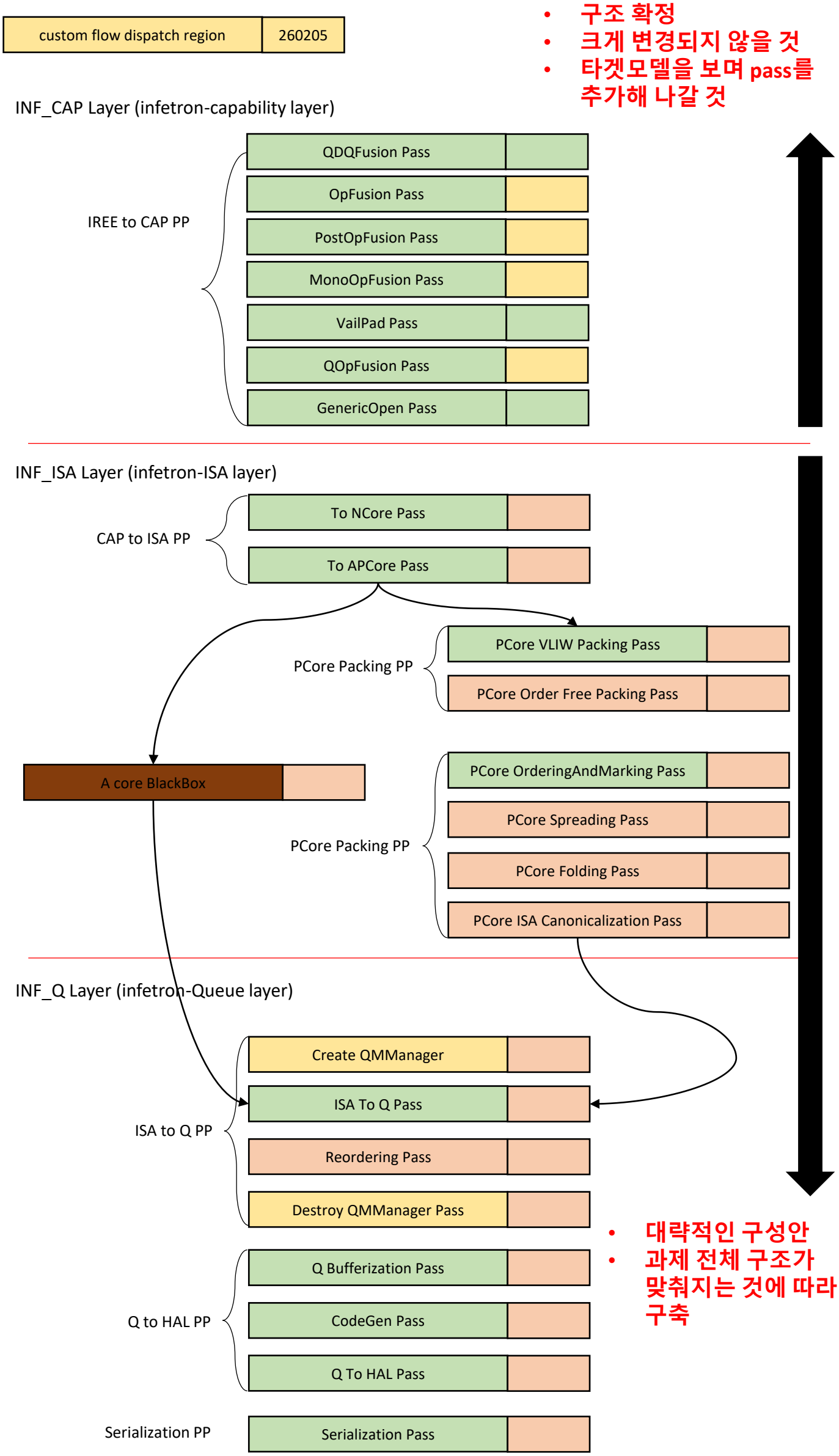
ISA to Q PP

- Create QMManager
- ISA To Q Pass
- Reordering Pass
- Destroy QMManager Pass

Q to HAL PP

- Q Bufferization Pass
- CodeGen Pass
- Q To HAL Pass

Serialization PP

- Serialization Pass

Workload (old)